model_training

May 12, 2026

[1]:
```
# pyright: reportUnusedImport=false, reportMissingImports=false,␣
 ↪reportMissingModuleSource=false
import pandas as pd
import numpy as np
import tensorflow as tf
import os
import seaborn as sb
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten,␣
 ↪Dropout, Rescaling
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
```

```
2026-05-12 23:38:48.390092: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not
find cuda drivers on your machine, GPU will not be used.
2026-05-12 23:38:48.467127: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to
register cuDNN factory: Attempting to register factory for plugin cuDNN when one
has already been registered
2026-05-12 23:38:48.467168: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
2026-05-12 23:38:48.467218: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
2026-05-12 23:38:48.481175: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not
find cuda drivers on your machine, GPU will not be used.
2026-05-12 23:38:48.482302: I tensorflow/core/platform/cpu_feature_guard.cc:182]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2026-05-12 23:38:52.219449: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not
```

```
find TensorRT
```

[2]:
```
normal_cells=os.listdir('brain_tumor_dataset/no/')
tumor_cells=os.listdir('brain_tumor_dataset/yes/')
```

1 total images in each folder

[3]:
```
print('Number of normal cells: ',len(normal_cells))
print('Number of tumor cells: ',len(tumor_cells))
```

```
Number of normal cells:  98
Number of tumor cells:  157
```

[4]:
```
TRAIN_DIR='brain_tumor_dataset'
```

2 image resize

[5]:
```
# Instantiate the dataset
train_dataset = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    image_size=(128, 128),
    batch_size=32,
    label_mode='binary',
    seed=100
)

# Check the type
dataset_type = type(train_dataset)
print(f'train_dataset inherits from tf.data.Dataset: {issubclass(dataset_type,␣
 ↪tf.data.Dataset)}')
```

```
Found 255 files belonging to 2 classes.
```

```
train_dataset inherits from tf.data.Dataset: True
```

[6]:
```
val_ds = tf.keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset='validation',
    image_size=(128, 128),
    batch_size=32,
    label_mode='binary',
    seed=100
)
dataset_type = type(train_dataset)
print(f'train_dataset inherits from tf.data.Dataset: {issubclass(dataset_type,␣
 ↪tf.data.Dataset)}')
```

```
Found 255 files belonging to 2 classes.
Using 51 files for validation.
train_dataset inherits from tf.data.Dataset: True
```

3 validating the image shapes

[7]:
```
# check the image shape and labels
for images, labels in train_dataset.take(1):
    print(f'Image batch shape: {images.shape}')
    print(f'Label batch shape: {labels.shape}')
```

```
2026-05-12 23:40:19.632280: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83]
Allocation of 19926816 exceeds 10% of free system memory.

Image batch shape: (32, 128, 128, 3)
Label batch shape: (32, 1)
```

[8]:
```
# genrate augmented images
data_augumentation=ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    vertical_flip=True,
    horizontal_flip=True
)
```

[9]:
```
rescale=Rescaling(1./255)
train_dataset = train_dataset.map(lambda x, y: (rescale(x), y))
val_ds = val_ds.map(lambda x, y: (rescale(x), y))
```

4 improving performance

[10]:
```
train_dataset=(
    train_dataset.cache()
    .shuffle(100).
    prefetch(buffer_size=tf.data.AUTOTUNE)
)
```

[11]:
```
val_ds=(
    val_ds.cache()
    .prefetch(buffer_size=tf.data.AUTOTUNE)
)
```

5 defining early stop at 82 percent

[12]:
```
class EarlyStopping(tf.keras.callbacks.Callback):
    def on_epoch_end(self,epoch,logs=None):
        if logs.get("accuracy")>0.82:
            print("\nReached 82% accuracy so cancelling training!")
            self.model.stop_training=True
```

[13]:
```
model=Sequential([
    tf.keras.layers.Input(shape=(128,128,3)),
    Conv2D(16,3,activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(32,3,activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(64,3,activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128,3,activation='relu'),
    MaxPooling2D(2,2),

    tf.keras.layers.Flatten(),
    Dense(256,activation='relu'),
    Dense(1,activation='sigmoid')


])
```

[14]:
```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 126, 126, 16)      448

 max_pooling2d (MaxPooling2  (None, 63, 63, 16)        0
 D)

 conv2d_1 (Conv2D)           (None, 61, 61, 32)        4640

 max_pooling2d_1 (MaxPoolin  (None, 30, 30, 32)        0
 g2D)

 conv2d_2 (Conv2D)           (None, 28, 28, 64)        18496

 max_pooling2d_2 (MaxPoolin  (None, 14, 14, 64)        0
 g2D)

 conv2d_3 (Conv2D)           (None, 12, 12, 128)       73856
```

```
 max_pooling2d_3 (MaxPoolin  (None, 6, 6, 128)         0
 g2D)

 flatten (Flatten)           (None, 4608)              0

 dense (Dense)               (None, 256)               1179904

 dense_1 (Dense)             (None, 1)                 257

=================================================================
Total params: 1277601 (4.87 MB)
Trainable params: 1277601 (4.87 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

[15]:
```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
    metrics=['accuracy']
)
```

6 using pre-build early stopping

using validation set also for fitting

[16]:
```
history = model.fit(
    train_dataset,
    validation_data=val_ds,
    epochs=10,
    callbacks=[
        tf.keras.callbacks.EarlyStopping(
            monitor='val_accuracy',
            patience=5,
            restore_best_weights=True
        )
    ]
    )
```

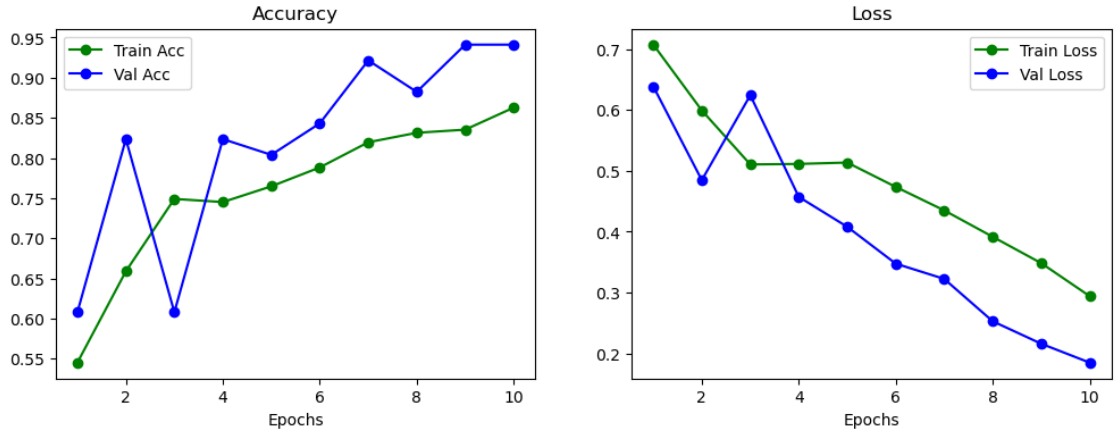
```
Epoch 1/10

2026-05-12 23:40:40.773818: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83]
Allocation of 19926816 exceeds 10% of free system memory.
2026-05-12 23:40:41.074132: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83]
Allocation of 32514048 exceeds 10% of free system memory.
2026-05-12 23:40:41.273091: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83]
Allocation of 24385536 exceeds 10% of free system memory.
2026-05-12 23:40:41.277263: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83]
```

```
Allocation of 24385536 exceeds 10% of free system memory.
8/8 [==============================] - 6s 393ms/step - loss: 0.7071 - accuracy:
0.5451 - val_loss: 0.6383 - val_accuracy: 0.6078
Epoch 2/10
8/8 [==============================] - 3s 342ms/step - loss: 0.5991 - accuracy:
0.6588 - val_loss: 0.4846 - val_accuracy: 0.8235
Epoch 3/10
8/8 [==============================] - 2s 306ms/step - loss: 0.5103 - accuracy:
0.7490 - val_loss: 0.6240 - val_accuracy: 0.6078
Epoch 4/10
8/8 [==============================] - 3s 350ms/step - loss: 0.5112 - accuracy:
0.7451 - val_loss: 0.4571 - val_accuracy: 0.8235
Epoch 5/10
8/8 [==============================] - 3s 346ms/step - loss: 0.5134 - accuracy:
0.7647 - val_loss: 0.4079 - val_accuracy: 0.8039
Epoch 6/10
8/8 [==============================] - 2s 294ms/step - loss: 0.4735 - accuracy:
0.7882 - val_loss: 0.3475 - val_accuracy: 0.8431
Epoch 7/10
8/8 [==============================] - 2s 291ms/step - loss: 0.4349 - accuracy:
0.8196 - val_loss: 0.3224 - val_accuracy: 0.9216
Epoch 8/10
8/8 [==============================] - 2s 290ms/step - loss: 0.3916 - accuracy:
0.8314 - val_loss: 0.2528 - val_accuracy: 0.8824
Epoch 9/10
8/8 [==============================] - 2s 294ms/step - loss: 0.3483 - accuracy:
0.8353 - val_loss: 0.2160 - val_accuracy: 0.9412
Epoch 10/10
8/8 [==============================] - 2s 294ms/step - loss: 0.2941 - accuracy:
0.8627 - val_loss: 0.1850 - val_accuracy: 0.9412
```

[18]:
```python
import matplotlib.pyplot as plt
try:
    h = history.history if hasattr(history, 'history') else history
    epochs = range(1, len(next(iter(h.values()))) + 1)
    plt.figure(figsize=(12,4))
    plt.subplot(1,2,1)
    if 'accuracy' in h or 'acc' in h:
        acc = h.get('accuracy') or h.get('acc')
        plt.plot(epochs, acc, 'go-', label='Train Acc')
    if 'val_accuracy' in h or 'val_acc' in h:
        val_acc = h.get('val_accuracy') or h.get('val_acc')
        plt.plot(epochs, val_acc, 'bo-', label='Val Acc')
    plt.title('Accuracy')
    plt.xlabel('Epochs')
    plt.legend()
    plt.subplot(1,2,2)
```

```
    if 'loss' in h:
        plt.plot(epochs, h['loss'], 'go-', label='Train Loss')
    if 'val_loss' in h:
        plt.plot(epochs, h['val_loss'], 'bo-', label='Val Loss')
    plt.title('Loss')
    plt.xlabel('Epochs')
    plt.legend()
    plt.show()
except Exception as e:
    print('Plot failed:', e)
    print('Ensure `history` is the return value from `model.fit(...)`')
```



[19]:
```
Test_Dir="test_images"
```

[32]:
```
test_dataset = tf.keras.utils.image_dataset_from_directory(
    Test_Dir,
    image_size=(128, 128),
    batch_size=32,
    label_mode='binary',
    seed=100
)
```

```
Found 255 files belonging to 2 classes.
```

[33]:
```
model.evaluate(test_dataset)
```

```
8/8 [==============================] - 1s 49ms/step - loss: 25.1839 - accuracy:
0.8745
```

[33]:
```
[25.18389129638672, 0.8745098114013672]
```

[34]:
```
model.predict(test_dataset)
```

```
8/8 [==============================] - 1s 44ms/step
```

[34]:
```
array([[1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [0.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.9278588e-23],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
       [0.0000000e+00],
       [1.0000000e+00],
       [1.0000000e+00],
```

```
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[9.9827162e-08],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[3.4573848e-12],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
```

```
[9.4065519e-28],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[8.1009492e-02],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[8.7972635e-01],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.2424599e-25],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
```

```
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.5089124e-36],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[8.7226500e-32],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.4556813e-28],
[4.0520990e-22],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
```

```
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[9.9995184e-01],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[4.0520990e-22],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
```

```
        [1.0000000e+00],
        [0.0000000e+00],
        [0.0000000e+00],
        [1.0000000e+00],
        [0.0000000e+00],
        [1.0000000e+00],
        [0.0000000e+00],
        [1.0000000e+00],
        [0.0000000e+00],
        [1.0000000e+00],
        [1.0000000e+00],
        [0.0000000e+00],
        [0.0000000e+00],
        [0.0000000e+00],
        [0.0000000e+00],
        [0.0000000e+00],
        [1.0000000e+00],
        [0.0000000e+00],
        [1.0000000e+00],
        [1.0000000e+00],
        [0.0000000e+00],
        [0.0000000e+00]], dtype=float32)
```

7 upload images

[ ]:
```python
import ipywidgets as widgets
from IPython.display import display, clear_output
import io
from PIL import Image


uploader = widgets.FileUpload(
    accept='image/*',   # Accept all image types
    multiple=False,     # Single file upload
    description='Upload Image',
    button_style='info' # Blue color
)
output = widgets.Output()

def on_predict_clicked(b):
    with output:
        clear_output()


        if not uploader.value:
            print(" Please upload an image first.")
```

```
            return

        print("Processing...")


        # Support both dict (new ipywidgets) and list/tuple formats
        if isinstance(uploader.value, dict):
            uploaded_file = next(iter(uploader.value.values()))
        else:
            uploaded_file = uploader.value[0]

        # filename may be at top-level or under metadata depending on widget/
 ↪version
        filename = uploaded_file.get('name') or (uploaded_file.get('metadata')␣
 ↪or {}).get('name', 'uploaded_image')
        content = uploaded_file.get('content') or uploaded_file.get('data')


        image = Image.open(io.BytesIO(content)).convert('RGB')
        display_img = image.copy()
        display_img.thumbnail((200, 200))


        image = image.resize((128, 128))
        image_array = np.array(image) / 255.0
        image_array = np.expand_dims(image_array, axis=0)


        prediction = model.predict(image_array, verbose=0)[0][0]
        label = 'Tumor' if prediction >= 0.5 else 'Normal'
        confidence = prediction if prediction >= 0.5 else 1 - prediction


        display(display_img)
        print(f'File: {filename}')
        print(f'Prediction: {label}')
        print(f'Confidence: {confidence:.2%}')


predict_btn = widgets.Button(
    description='Predict Tumor',
    button_style='success',
    tooltip='Click to analyze the uploaded image'
)
predict_btn.on_click(on_predict_clicked)
```

```
print("--- Brain Tumor Classifier ---")
display(widgets.VBox([uploader, predict_btn, output]))
```

```
--- Brain Tumor Classifier ---
```

```
VBox(children=(FileUpload(value=(), accept='image/*', button_style='info',␣
 ↪description='Upload Image'), Button…
```

7.0.1 wraping our model in .keras(.h5) to consume at backend fast api

[24]:
```
model.save('brain_tumor_classifier.keras')
```